

Hiring Assignment: Build a Browser AI Copilot

Objective

Build a AI Copilot as a Chrome Extension that can understand any webpage, interact via chat, and perform actions on behalf of the user.

The Copilot should behave like an agent, not just a chatbot.

Use Case

A user is browsing any website (CRM, dashboard, LinkedIn, docs, etc.) and uses the Copilot to:

- Summarize this page
- Extract all leads from this page
- Fill this form with dummy data
- Reply to this message professionally

The Copilot should:

- Understand the page
 - Answer questions
 - Take actions
-

Requirements

Setup

- Build a Google Chrome Extension (Manifest v3)
 - Use content scripts for DOM interaction
 - Backend: Node.js
 - Frontend: Vue.js (sidebar/chat UI)
-

Core Features

1. Chat-Based Copilot (Primary Interface)

- Sidebar UI
 - Multi-turn conversation
 - Maintains context of current page
-

2. Page Context Understanding

- Extract relevant data from DOM:
 - Text content
 - Forms / inputs
 - Buttons / tables
-

3. Reasoning & Assistance

Copilot should:

- Summarize content
 - Answer questions about the page
 - Provide recommendations
-

4. Action Capabilities (Agent Behavior)

From chat, Copilot can:

- Click elements
- Fill forms
- Extract structured data
- Draft replies

Example:

“Reply professionally” → auto-fill text field

5. Tool-Based Design (Important)

Implement clear tools such as:

- read_page
- click_element
- fill_input
- extract_data

Separate:

- Chat layer
 - Agent logic
 - Browser interaction
-

Deliverables

1. Code & Implementation

- Chrome Extension (Manifest v3)
 - Content scripts + background logic
 - Sidebar UI (Vue.js)
 - Backend (Node.js, if used)
 - Steps to install and run locally
-

2. Demo (2–5 Minutes)

Show:

- Copilot on a live webpage
 - Chat interaction
 - Page understanding (e.g., summarization)
 - Action execution (e.g., form fill / data extraction)
 - At least one multi-step task
-

3. Documentation (README)

Include:

- Architecture (extension + agent design)
- How page context is extracted

- Tool/action design
 - What's real vs mocked
 - Trade-offs
-

Evaluation Criteria

- Agent Design: Can reason and take actions (not just respond)
 - Browser Integration: DOM extraction + reliable actions
 - Product Thinking: Useful workflows via chat
 - UX Quality: Smooth, intuitive Copilot experience
 - Completeness: End-to-end working system
 - Code Quality: Modular and maintainable
-

Submission

- GitHub repo
- Demo video
- README